

FACTORY / WAREHOUSE / STORAGE MANAGEMENT SYSTEM

Complete Full-Stack Development Prompt

Build a **real, production-ready Factory / Warehouse / Storage Management System**.

This is NOT a static UI prototype.

The system must be fully connected:

- **Frontend:** React.js
- **Backend:** Node.js + Express.js
- **Database:** MySQL
- **Authentication:** Secure phone-number + password authentication
- **PDF:** Professional invoice/sales-document generation
- **Images:** Product/category/company logos and other required images
- **API:** RESTful API between frontend and backend
- **Responsive:** Desktop, tablet and mobile
- **Architecture:** Multi-factory / multi-warehouse capable

The system should be designed for businesses such as:

- Furniture factories
- Furniture warehouses
- Manufacturing companies
- Building-material warehouses
- Spare-parts warehouses
- Electronics warehouses
- General storage stores
- Wholesale businesses
- Distribution centers
- Large shops with inventory
- Production + warehouse businesses

The system must be configurable so that the same application can be used for different types of businesses.

1. SYSTEM CONCEPT

The application is a **business management and inventory system**.

Each factory, warehouse, or storage store has its own:

- Account
- Login

- Logo
- Name
- Address
- Phone numbers
- Business information
- Categories
- Products
- Inventory
- Orders
- Sales
- Customers
- Suppliers
- Employees/users
- Reports
- PDF templates
- Settings
- History
- Activity records

The **System Admin** controls the entire platform.

A factory/warehouse account only sees and manages its own business data.

Data from one factory/warehouse must NEVER accidentally appear in another factory/warehouse account.

2. ACCOUNT STRUCTURE

Use a multi-tenant architecture.

There should be a central:

SYSTEM ADMIN

The System Admin can:

- Create factory accounts
- Create warehouse accounts
- Create storage-store accounts
- Edit accounts
- Disable accounts
- Activate accounts
- Delete accounts where appropriate
- Reset passwords
- View business information
- View system statistics
- Control permissions
- Control system settings

- View activity logs
 - View reports
 - Manage system-wide configuration
 - Manage subscription/license information if needed
 - Manage backups
 - Monitor system health
 - Manage all factories and warehouses
-

3. FACTORY / WAREHOUSE ACCOUNT

When the System Admin creates a new business account, the admin enters:

Business information

- Business name
- Business type
- Business logo
- Business description
- Phone number
- Secondary phone number
- Email
- Address
- City
- Country
- Website
- Tax number if applicable
- Registration number if applicable
- Currency
- Default language
- Time zone
- Business status

Login information

- Phone number
- Password
- Confirm password

The system must securely hash passwords.

Never store plain-text passwords.

4. LOGIN SCREEN

The login screen should be professional and business-oriented.

When a factory/warehouse account is opened, display:

- Business logo
- Business name
- Optional business description
- Phone-number input
- Password input
- Show/hide password
- Login button
- Forgot password
- Remember session where appropriate

Example:

[BUSINESS LOGO]

ABC Furniture Factory

Phone Number

Password

LOGIN

The branding displayed after login should belong to that specific factory/warehouse.

5. DASHBOARD

After login, show a dashboard customized for that business.

Dashboard should include:

Main statistics

- Total products
- Total categories
- Total stock quantity
- Low-stock products
- Out-of-stock products
- Today's sales
- Today's orders
- This month's sales
- Total sales
- Pending orders
- Completed orders
- Cancelled orders
- Returned orders
- Total customers
- Total suppliers

Visual reports

Use charts for:

- Daily sales
- Weekly sales
- Monthly sales
- Yearly sales
- Product sales
- Category sales
- Stock movement
- Purchases
- Returns
- Profit if cost prices are available

Quick actions

Buttons such as:

- Add Product
- Add Category
- New Sale
- New Order
- Add Stock
- Add Customer
- Add Supplier
- Generate Report

6. SIDEBAR / MAIN NAVIGATION

The main navigation should contain:

1. Dashboard
2. Products
3. Categories
4. Inventory
5. Sales
6. Orders
7. Customers
8. Suppliers
9. Purchases
10. Returns
11. Production
12. Employees / Users
13. Reports
14. Documents
15. Activity History
16. Settings

The system can hide modules that are disabled for a particular business.

7. CATEGORIES

The business can create unlimited categories.

Example for furniture:

- Sofas
- Beds
- Tables
- Chairs
- Wardrobes
- Cabinets
- Office Furniture
- Kitchen Furniture

For another business, categories can be completely different.

Each category can contain:

- Category name
- Category image
- Description
- Parent category
- Category code
- Status
- Sort order
- Created date
- Updated date

Support:

- Add category
 - Edit category
 - Delete category
 - Activate/deactivate category
 - Search
 - Filter
 - Parent/subcategory structure
-

8. PRODUCTS

Create a complete product management system.

Each product should support:

Basic information

- Product name
- Product code
- SKU
- Barcode
- Category
- Subcategory
- Brand
- Product image
- Multiple images
- Description
- Short description
- Status

Inventory information

- Current quantity
- Minimum stock
- Maximum stock
- Reorder level
- Reserved quantity
- Available quantity
- Warehouse/location
- Shelf/rack/bin
- Unit of measurement

Examples:

- Piece
- Box
- Set
- Meter
- Kilogram
- Liter
- Square meter
- Cubic meter

Financial information

- Purchase cost
- Selling price
- Wholesale price
- Discount price
- Tax
- Profit margin

Optional information

- Size
- Length
- Width

- Height
- Weight
- Color
- Material
- Model
- Manufacturer
- Serial number
- Batch number
- Expiry date
- Warranty period

Do not force unnecessary fields.

The product form should adapt to the business type.

9. PRODUCT VARIANTS

Products may have variants.

Example:

Sofa

- Color: Black
- Color: White
- Color: Brown

Or:

Table

- Size: 120cm
- Size: 160cm
- Size: 200cm

Each variant can have:

- SKU
 - Barcode
 - Price
 - Cost
 - Quantity
 - Images
 - Attributes
-

10. INVENTORY MANAGEMENT

Inventory must be one of the most important modules.

Track:

- Stock received
- Stock sold
- Stock returned
- Stock transferred
- Stock adjusted
- Damaged stock
- Reserved stock
- Available stock

Every inventory change must create a history record.

Example:

Product: Office Chair

Previous Stock: 100

Sold: 5

New Stock: 95

Record:

- Date
 - Time
 - User
 - Action
 - Quantity
 - Reason
 - Reference order
-

11. MULTIPLE STORAGE LOCATIONS

The business may have multiple locations.

Example:

- Main Warehouse
- Warehouse A
- Warehouse B
- Showroom
- Production Area
- Storage Room

- Outdoor Storage

Products can be assigned to locations.

Support:

- Stock transfer
- Location-to-location transfer
- Transfer history
- Transfer status
- Transfer notes

12. STOCK MOVEMENTS

Create a dedicated stock movement history.

Types:

- Purchase
- Sale
- Return
- Damage
- Adjustment
- Transfer
- Production
- Manual increase
- Manual decrease

Every movement must have:

- Product
- Quantity
- Previous quantity
- New quantity
- Movement type
- User
- Date/time
- Location
- Note
- Reference number

Records should never silently disappear.

13. SALES

Create a complete sales system.

The user can create a sale by selecting:

- Customer
- Product
- Quantity
- Price
- Discount
- Tax
- Notes

Automatically calculate:

- Subtotal
- Discount
- Tax
- Shipping/extra charges if enabled
- Grand total
- Paid amount
- Remaining amount

Payment status:

- Paid
- Partially Paid
- Unpaid

Payment methods should be configurable:

- Cash
- Bank transfer
- Card
- Other

14. ORDER SYSTEM

Every sale should have an order/document number.

Example:

ORD-2026-000001

Order statuses:

- Draft
- Pending
- Confirmed
- Processing
- Ready
- Completed

- Cancelled
- Returned
- Partially Returned

The business must be able to:

- Create order
 - Edit order
 - Add product
 - Remove product
 - Change quantity
 - Change price if permission allows
 - Add discount
 - Add customer
 - Add notes
 - Change status
 - Cancel order
 - Return products
 - Reopen where permitted
 - Print
 - Generate PDF
-

15. ORDER EDIT HISTORY

Never silently overwrite important financial/order information.

When an order is edited, record:

- User
- Date
- Time
- Previous value
- New value
- Reason if required

Example:

Quantity changed:

10 → 8

Changed by:

Admin User

Date/time:

2026-09-16 14:20

16. RETURNS

The system must support product returns.

Users can return:

- Entire order
- Selected products
- Partial quantity

Return information:

- Original order
- Customer
- Product
- Quantity
- Return reason
- Condition
- Refund amount
- User
- Date
- Notes

Return status:

- Requested
- Approved
- Rejected
- Completed

Inventory must automatically update according to the configured return process.

17. CANCELLATIONS

Orders can be cancelled.

Cancellation should record:

- Order number
- User
- Date/time
- Reason
- Previous status
- New status

The system should prevent accidental cancellation with confirmation.

18. CUSTOMERS

Although this is primarily a factory/warehouse management system, customers should be supported because sales need customer information.

Customer fields:

- Full name
- Phone
- Secondary phone
- Email
- Address
- Company
- Notes
- Customer code
- Total purchases
- Outstanding balance
- Order history

Customer information can be optional for quick sales if the business allows anonymous/cash sales.

19. SUPPLIERS

Supplier management should include:

- Supplier name
- Company
- Phone
- Email
- Address
- Contact person
- Products supplied
- Purchase history
- Outstanding balance
- Notes

20. PURCHASES

Support purchasing stock from suppliers.

Purchase fields:

- Supplier
- Purchase number
- Products
- Quantity

- Cost
- Discount
- Tax
- Total
- Paid amount
- Remaining balance
- Payment method
- Date
- Notes

When a purchase is completed:

Inventory automatically increases.

21. PRODUCTION MODULE

For factories, include an optional production module.

This should support:

Raw materials

- Material name
- SKU
- Quantity
- Unit
- Cost
- Supplier
- Minimum stock

Finished products

- Product
- Quantity produced
- Production date
- Production batch
- Production cost

Bill of Materials

Example:

One table requires:

- Wood: 5 pieces
- Screws: 20
- Paint: 1 liter
- Metal legs: 4

When production is completed:

- Raw materials decrease
- Finished goods increase
- Production record is created

Production module should be optional for warehouses that do not manufacture products.

22. FACTORY PRODUCTION HISTORY

Track:

- Production number
- Product
- Quantity
- Materials used
- Production cost
- Employee/team
- Production date
- Start time
- Completion time
- Status
- Notes

Statuses:

- Planned
 - In Progress
 - Completed
 - Cancelled
-

23. EMPLOYEES / USERS

The System Admin can create users for each factory/warehouse.

Do NOT give every user full access.

Use role-based permissions.

Example roles:

Business Owner/Admin

Full access to their business.

Manager

Can manage most business operations.

Warehouse Manager

Inventory, products, transfers, stock.

Sales Staff

Sales and customers.

Inventory Staff

Stock operations.

Production Manager

Production and materials.

Accountant

Financial records and reports.

Viewer

Read-only access.

Permissions should be configurable.

24. PERMISSION SYSTEM

Permissions should exist at module/action level.

Examples:

- View Products
- Create Products
- Edit Products
- Delete Products
- View Inventory
- Adjust Inventory
- Create Sales
- Edit Sales
- Cancel Sales
- Process Returns
- View Reports
- Export Reports

- Manage Users
- Manage Settings
- View Financial Information

Support:

- View
 - Create
 - Edit
 - Delete
 - Approve
 - Export
-

25. REPORTS

Create a powerful Reports section.

Reports should include:

Sales reports

- Daily sales
- Weekly sales
- Monthly sales
- Yearly sales
- Custom date range
- Sales by product
- Sales by category
- Sales by customer
- Sales by employee

Inventory reports

- Current inventory
- Low stock
- Out of stock
- Stock movement
- Stock valuation
- Damaged products
- Transfer history

Purchase reports

- Purchases by date
- Purchases by supplier
- Purchase costs

Returns

- Return history

- Returned products
- Return amounts

Financial

If financial data is enabled:

- Revenue
- Cost
- Gross profit
- Discounts
- Tax
- Outstanding payments

Every report should support:

- Date filter
 - Product filter
 - Category filter
 - User filter
 - Location filter
 - Search
 - Export PDF
 - Export Excel/CSV where appropriate
-

26. TWO MAIN REPORT AREAS

Create two major report dashboards.

Operational Reports

Show:

- Inventory
- Stock movement
- Products
- Production
- Purchases
- Transfers
- Returns

Business Reports

Show:

- Sales
- Revenue
- Costs
- Profit

- Customers
- Suppliers
- Payments
- Outstanding balances

This separation makes the system easier to understand.

27. PDF SALES DOCUMENT

After completing a sale, the system must be able to generate a professional PDF.

The PDF should include:

Business header

- Logo
- Business name
- Business description
- Address
- Phone
- Email
- Website
- Tax/registration information if configured

Sale information

- Invoice/order number
- Date
- Time
- Customer name
- Customer phone
- Customer address if available
- Employee/user

Products table

- Product
- SKU
- Quantity
- Unit
- Unit price
- Discount
- Tax
- Total

Summary

- Subtotal
- Discount
- Tax

- Extra charges
- Grand total
- Paid
- Remaining

Footer

- Notes
 - Return policy
 - Payment terms
 - Thank-you message
 - Authorized signature area
-

28. PDF TEMPLATE BUILDER

In Settings, the business should be able to customize the PDF.

Settings should include:

- Logo
- Business name
- Description
- Address
- Phone
- Email
- Website
- Footer text
- Thank-you message
- Return policy
- Payment terms
- Signature text
- Invoice title
- Currency
- Date format

Allow enabling/disabling fields.

For example:

- ☒ Logo
- ☒ Phone
- ☒ Address
- ☒ Customer phone
- ☐ Customer address
- ☒ Tax
- ☒ Signature

The generated PDF must use the selected template.

29. DOCUMENT NUMBERING

Allow configurable numbering.

Examples:

- INV-000001
- SALE-000001
- ORD-000001
- RET-000001
- PUR-000001

Allow business administrators to configure prefixes and starting numbers.

30. ACTIVITY HISTORY / AUDIT LOG

Create a complete audit system.

Record important actions:

- Login
- Logout
- Product created
- Product edited
- Product deleted
- Category created
- Stock changed
- Sale created
- Sale edited
- Sale cancelled
- Return created
- Purchase created
- User created
- Permission changed
- Settings changed

Record:

- User
- Action
- Module
- Description
- Date
- Time
- IP where appropriate
- Reference ID

Provide search and filters.

31. NOTIFICATIONS

Create an internal notification system.

Examples:

- Low stock
- Out of stock
- New order
- Return request
- Pending payment
- Production completed
- Transfer received
- Important system alert

Notifications should be configurable.

32. SEARCH

Provide global and module-specific search.

Search products by:

- Name
- SKU
- Barcode
- Product code

Search orders by:

- Order number
- Customer
- Phone

Search customers by:

- Name
- Phone

Search suppliers by:

- Name
 - Phone
-

33. BARCODE SUPPORT

Support barcode scanning where applicable.

The system should allow:

- Barcode field
- Barcode search
- Barcode scanning from compatible devices
- Generate product barcode
- Print barcode labels

Barcode functionality should be optional.

34. SETTINGS

Create a complete Settings module.

Sections:

Business Settings

- Business information
- Logo
- Contact information
- Address
- Currency
- Tax

User Settings

- Users
- Roles
- Permissions

Inventory Settings

- Units
- Low-stock rules
- Stock adjustment rules
- Locations

Sales Settings

- Invoice numbering
- Payment methods
- Discounts
- Tax

PDF Settings

- Invoice template
- Logo
- Header
- Footer
- Signature

Production Settings

- Production numbering
- Units
- Material rules

Security

- Password policy
 - Session management
 - Login protection
-

35. SECURITY

Implement professional security.

Backend must include:

- Password hashing
- Authentication tokens/session security
- Authorization middleware
- Role-based access control
- Input validation
- SQL injection protection
- Rate limiting
- Secure HTTP headers
- CORS configuration
- Request validation
- Error handling
- Audit logs

Never trust frontend permissions.

All important authorization must be enforced on the Node.js backend.

36. MULTI-TENANT DATA SECURITY

This is extremely important.

Every business record must contain the appropriate:

- business_id / factory_id / tenant_id

Backend queries must always enforce tenant isolation.

Example:

A user from Factory A must NEVER be able to request:

Factory B products
Factory B orders
Factory B customers
Factory B inventory
Factory B reports

System Admin can access all businesses.

Business users can only access their own business.

37. MYSQL DATABASE

Design a normalized MySQL database.

Recommended tables include:

- users
- roles
- permissions
- role_permissions
- businesses
- business_settings
- categories
- products
- product_variants
- product_images
- units
- warehouses
- storage_locations
- inventory
- inventory_movements
- customers
- suppliers
- purchases
- purchase_items
- orders
- order_items
- payments

- returns
- return_items
- production_orders
- production_items
- bill_of_materials
- stock_transfers
- notifications
- pdf_templates
- audit_logs
- system_settings

Use:

- Primary keys
- Foreign keys
- Indexes
- Unique constraints
- Timestamps
- Soft deletion where appropriate

38. API ARCHITECTURE

Create clean REST APIs.

Example:

Authentication

POST /api/auth/login
 POST /api/auth/logout
 POST /api/auth/change-password

Businesses

GET /api/businesses
 POST /api/businesses
 GET /api/businesses/:id
 PUT /api/businesses/:id
 DELETE /api/businesses/:id

Products

GET /api/products
 POST /api/products
 GET /api/products/:id
 PUT /api/products/:id
 DELETE /api/products/:id

Categories

GET /api/categories
POST /api/categories
PUT /api/categories/:id
DELETE /api/categories/:id

Orders

GET /api/orders
POST /api/orders
GET /api/orders/:id
PUT /api/orders/:id
POST /api/orders/:id/cancel
POST /api/orders/:id/return

Inventory

GET /api/inventory
POST /api/inventory/adjust
POST /api/inventory/transfer
GET /api/inventory/movements

Reports

GET /api/reports/sales
GET /api/reports/inventory
GET /api/reports/purchases
GET /api/reports/returns

Follow consistent API response formats.

39. FRONTEND STRUCTURE

Use React.js.

Recommended structure:

src/

- components/
- pages/
- layouts/
- features/
- services/
- hooks/
- context/
- utils/

- types/
- routes/
- assets/

Use reusable components.

Do not duplicate UI logic.

40. RESPONSIVE DESIGN

The application must work properly on:

- Desktop
- Laptop
- Tablet
- Mobile

Mobile screens should have:

- Responsive sidebar
- Mobile navigation
- Responsive tables
- Touch-friendly buttons
- Responsive forms
- Mobile-friendly PDF preview
- Mobile-friendly dashboard

The user should be able to manage important operations from a phone.

41. UI/UX

Design should look like a modern professional business application.

Use:

- Clean dashboard
- Clear navigation
- Professional tables
- Search
- Filters
- Pagination
- Confirmation dialogs
- Toast notifications
- Loading states
- Empty states
- Error states
- Skeleton loading where useful

- Form validation

Do not create a flashy design that makes business operations difficult.

Prioritize:

Speed + clarity + usability + reliability.

42. PRODUCT TABLE

Product table should show:

- Image
- Product name
- SKU
- Category
- Quantity
- Location
- Cost
- Selling price
- Status
- Actions

Actions:

- View
 - Edit
 - Stock
 - History
 - Delete/archive
-

43. ORDER TABLE

Order table:

- Order number
- Customer
- Date
- Items
- Total
- Paid
- Remaining
- Status
- Created by
- Actions

Actions:

- View
- Edit
- PDF
- Print
- Return
- Cancel
- History

44. INVENTORY ALERTS

Automatically detect:

Low Stock

Current quantity $\leq$ reorder level.

Out of Stock

Quantity = 0.

Overstock

Quantity > configured maximum.

Show these on the dashboard.

45. SOFT DELETE

Do not immediately permanently delete important business records.

For important records use:

- active
- archived
- deleted_at

Historical sales and financial records should remain consistent.

46. DATABASE TRANSACTIONS

Use MySQL transactions for operations that modify multiple records.

Example sale:

1. Create order
2. Create order items
3. Update inventory
4. Create inventory movement
5. Create payment
6. Create audit log

If one critical step fails, rollback the transaction.

Never leave the database in a partially updated state.

47. STOCK VALIDATION

Prevent selling more stock than available unless the business explicitly enables negative inventory.

Example:

Available:

5

User tries to sell:

8

Show:

Insufficient stock. Available quantity: 5.

Allow negative inventory only through an explicit business setting.

48. REPORT EXPORT

Reports should support:

- PDF
- Excel/CSV where appropriate

Exported reports should contain:

- Business logo
- Business name
- Report title
- Date range

- Filters
 - Data
 - Totals
 - Generated date
 - Generated by
-

49. DASHBOARD CUSTOMIZATION

Business admin should be able to configure which dashboard cards are displayed.

Possible widgets:

- Sales
 - Inventory
 - Low stock
 - Customers
 - Purchases
 - Production
 - Returns
 - Profit
 - Payments
-

50. FACTORY TYPE CONFIGURATION

When creating a business, allow the System Admin to select:

- Furniture Factory
- General Factory
- Warehouse
- Storage Store
- Wholesale Store
- Distribution Center
- Custom

The system can enable relevant modules automatically.

Example:

Furniture Factory:

Products + Inventory + Sales + Purchases + Production + Raw Materials

Warehouse:

Products + Inventory + Transfers + Purchases + Sales

Storage Store:

Products + Inventory + Sales + Customers

51. BUSINESS CUSTOM FIELDS

Allow optional custom fields.

For example, a furniture factory might need:

- Material
- Wood type
- Fabric type
- Color
- Dimensions

A spare-parts warehouse might need:

- Vehicle model
- Part number
- Manufacturer

A building-material store might need:

- Length
- Width
- Thickness
- Weight

The system should support configurable custom fields rather than hard-coding every industry.

52. BACKUP AND RESTORE

The system should be designed for database backups.

Support:

- Scheduled backup
- Manual backup
- Backup history
- Restore procedure

Only authorized administrators can access backup functions.

53. ERROR HANDLING

Use centralized backend error handling.

Never expose sensitive backend information to users.

Frontend should show understandable messages.

Example:

Instead of:

```
SequelizeDatabaseError...
```

Show:

Unable to save the product. Please try again.

Log technical details on the server.

54. VALIDATION

Validate both frontend and backend.

Required fields must be clearly marked.

Examples:

- Business name
- Phone number
- Password
- Product name
- Product code where required
- Quantity
- Price

Prevent invalid:

- Negative quantities
 - Invalid prices
 - Duplicate SKUs
 - Duplicate document numbers
 - Invalid user permissions
-

55. AUDITABILITY

Important records must always be traceable.

For a sale, the system should be able to answer:

- Who created it?
 - When?
 - Which products?
 - What quantities?
 - What prices?
 - What discount?
 - What payment?
 - Was it edited?
 - Was it cancelled?
 - Was it returned?
 - Who performed each action?
-

56. ADMIN SYSTEM DASHBOARD

The global System Admin dashboard should show:

- Total businesses
- Active businesses
- Disabled businesses
- Total users
- Total products
- Total orders
- Total sales across system
- Recent activity
- Recently created businesses
- System alerts

Admin can open a business and inspect its system.

57. BUSINESS DETAILS PAGE FOR SYSTEM ADMIN

When System Admin opens a factory/warehouse:

Show:

Business profile

- Logo
- Name
- Type

- Address
- Phone
- Email
- Status

Statistics

- Products
- Categories
- Inventory
- Orders
- Sales
- Customers
- Suppliers
- Users

Activity

- Recent logins
- Recent sales
- Recent changes
- Recent errors

Controls

- Edit
- Disable
- Activate
- Reset password
- Manage users
- View reports

58. SESSION AND LOGIN SECURITY

Implement:

- Secure login
- Session expiration
- Logout
- Password change
- Login attempt protection
- Account lock/rate limit where appropriate

The system should never reveal whether a specific phone number exists during password recovery in a way that leaks account information.

59. PERFORMANCE

The application must be designed for large amounts of data.

Use:

- Pagination
- Database indexes
- Efficient SQL queries
- Server-side filtering
- Server-side sorting
- Lazy loading
- Optimized images
- Caching where appropriate

Do NOT load thousands of products into the browser unnecessarily.

60. SCALABILITY

The architecture should allow the system to grow from:

10 products → 10,000+ products

10 orders → millions of historical records

1 warehouse → multiple warehouses

1 business → many businesses

without requiring a complete rewrite.

61. IMPORTANT BUSINESS RULES

Implement these rules carefully:

1. Inventory must update automatically after completed sales.
2. Inventory must increase after completed purchases.
3. Returns must correctly affect inventory.
4. Cancelled orders must not incorrectly remain as completed sales.
5. Editing orders must preserve history.
6. Important financial records must not be silently deleted.
7. Every business must be isolated.
8. Every important action must be auditable.
9. Permissions must be enforced by the backend.
10. Reports must use reliable database calculations.

11. PDF documents must use the business's configured template.
 12. Document numbers must be unique.
 13. Stock cannot become negative unless explicitly enabled.
 14. Database transactions must protect multi-step operations.
-

62. TECHNOLOGY REQUIREMENTS

Frontend

Use:

- React.js
- React Router
- Modern component architecture
- API service layer
- Form validation
- Responsive CSS
- Professional dashboard UI

Backend

Use:

- Node.js
- Express.js
- REST API
- Authentication middleware
- Authorization middleware
- Validation middleware
- Centralized error handling
- Logging

Database

Use:

- MySQL
 - Proper relational schema
 - Foreign keys
 - Indexes
 - Transactions
-

63. CODE QUALITY

The code must be:

- Modular
- Maintainable
- Reusable
- Secure
- Well structured
- Properly commented where necessary
- Easy to extend

Avoid:

- Giant files
 - Duplicate logic
 - Hard-coded business data
 - Hard-coded permissions
 - Hard-coded tenant IDs
 - Frontend-only security
 - Fake API responses
 - Mock data pretending to be real data
-

64. NO STATIC PROTOTYPE

The final application must NOT be a fake dashboard.

Every major button must perform a real action.

Examples:

Add Product

→ saves to MySQL.

Create Sale

→ creates real order + updates inventory.

Return Product

→ creates real return + updates inventory.

Generate PDF

→ generates a real PDF.

Create Business

→ creates a real business account.

Login

→ authenticates against the Node.js backend.

Reports

→ retrieve real database data.

65. SEED / DEMO DATA

Create optional development seed data.

Example business:

ABC Furniture Factory

Categories:

- Sofas
- Tables
- Chairs
- Beds
- Wardrobes

Products:

- 3-Seater Sofa
- Dining Table
- Office Chair
- King Bed
- Wooden Wardrobe

But clearly separate demo/seed data from production data.

66. FINAL SYSTEM FLOW

The complete flow should be:

System Admin

Login

↓

Admin Dashboard

↓

Create Factory / Warehouse

↓

Enter business information

↓

Upload logo

↓

Create phone + password

↓

Business account created

↓

Business logs in

↓

Business Dashboard

↓

Configure settings

↓

Create categories

↓

Create products

↓

Configure locations

↓

Add initial inventory

↓

Create customers/suppliers

↓

Create purchases or production

↓

Inventory updates

↓

Create sale/order

↓

Inventory updates

↓

Generate PDF

↓

Customer receives document

↓

Order can later be:

- Edited
- Cancelled
- Returned

↓

All actions stored in history

↓

Reports calculate real data

67. FINAL QUALITY REQUIREMENT

Build this as a **real business management product**, not as a simple inventory CRUD application.

The system should feel like professional software that a real factory, warehouse, storage store, or wholesale business could use every day.

Prioritize:

1. Data correctness
2. Security
3. Tenant isolation
4. Inventory accuracy
5. Order accuracy
6. Audit history
7. Professional PDF documents
8. Reporting
9. Responsive design
10. Maintainability
11. Performance
12. Scalability

Before considering the project complete, test:

- Login
- Permissions
- Business isolation
- Product CRUD
- Category CRUD
- Inventory
- Purchases
- Sales
- Orders
- Order editing
- Cancellation
- Returns
- Payments
- PDF generation
- Reports
- Users
- Permissions
- Settings
- Audit logs
- Mobile responsiveness
- Error handling
- Database transactions

The final result must be a **fully functional React + Node.js + MySQL Factory / Warehouse / Storage Management System** with real authentication, real database operations, real inventory tracking, real sales/order management, real reports, real history, and professional PDF document generation.